

Tests For The Hangman Game

Name: Fredrik Strömbäck

LNU-email: fs222uv@student.lnu.se

GitHub repo: https://github.com/Strombach/fs222uv_1dv600

Release: https://github.com/Strombach/fs222uv_1dv600/releases/tag/4.0

1. Test Plan

What are the objectives of the testing in this iteration?

The objective in this last iteration is to expand my tests with more test for the new feature I added. The new feature is a high-score list. But all the other test suites are still there to be tested as well. So I will expand this document for the last iteration.

What to test and how?

For the manual tests I'm going to test UC3 where the gamer should be able to quit the game from anywhere in the game, after confirming to do so. The other part to test is where the gamer can enter his/her name in UC2 "Play Game", this is the first five steps in the main scenario of UC2. I will also test if the gamer enters a name that is too short or too long, because the game only accepts a name that is three character long.

The manual tests for iteration 4 will be added to test UC4 "Check High-Scores", here the tests are to see that the gamer return to menu or quit the game from the list. Even to see that the module can read the file highscores.json.

I don't feel that UC1 "Start Game" needs to have any test cases because it is always automatically tested in the other manual test cases and are necessary to work for the other test cases to work.

These tests will be running dynamically so starting the application is necessary.

The automated unit tests will be testing the constructor for the Gamer class to see that the class is initiated correctly and throws exceptions when the input is wrong. The createLetter method in the Word class will be tested to see that the app can create individual letter objects. Both the Gamer constructor and Word-class methods will have at least two tests each.

I will be using the framework Mocha with Chai assertion library for all the automated unit tests. To start automated test, enter "npm run test" in the console.

Use Cases To Be Tested

Use Case 2 Play Game

Preconditions: The game is running and the main menu is displayed.

Main success scenario (Basic Flow)

1. The gamer chooses the option “Play game” in the menu.*
2. The system ask for a name that the user wants to have in the game.*
3. The gamer enters a name.*
4. The system picks a word from a word list.*
5. The system displays info about the gamer and the word with the letters hidden.*
6. The gamer guesses a letter.
7. The system replaces one or more of the underlines, where the guessed letter matches the hidden letter, with the correctly guessed letters.
8. The gamer guesses all the letters correctly.
9. The system displays that the gamer have won and returns to the menu.

Alternative Flow

3a. The gamer enters a name that is too short or too long. *

1. The system displays an error message and rejects the entry.
2. Go to 2.

* = Part to be tested

Use Case 3 Quit Game

Precondition: The game is running.

Post condition: The game is terminated.

Main scenario

- 1.Starts when the user wants to quit the game.
- 2.The system prompts for confirmation.
- 3.The user confirms.
- 4.The system terminates.

Alternative scenarios

3.1. The user does not confirm

- 1.The system returns to its previous state

UC 4 Check High-Scores

Precondition: The game is running and the menu is shown.

Postcondition: The high-score list is shown.

Main scenario

1. The gamer chooses the high-score option.
2. The system fetches list from a file and displays it in the console.
3. The gamer chooses to go back to the menu.
4. The system enters the main menu again.

Extensions (Alternatives Flows)

2a. The high-score file is empty.

1. The system prompts a message about empty list.
2. The gamer chooses to go back to the main menu.
3. The system enters the main menu.

2b. There is no high-score file.

1. The system creates a json file to keep the high-scores in.
2. Go to alternative flow 2a.

3a. The gamer wants quits the game

1. The gamer enters command for quitting the game.
2. The systems quits the game (see Use Case 3)

Test Cases

TC1.1 Name of the gamer is stored successfully.

This is tested to see that game can take and store a name into the gamer class.

Use Case: First five steps of UC2 “Play Game”

Scenario: The name is stored successfully.

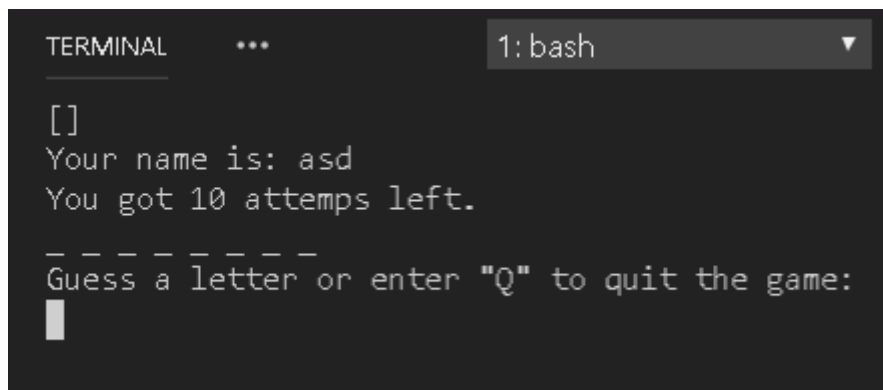
Precondition: The app is running and user can see the menu.

Test steps

1. Press number 1 and press enter.
2. System shows “Enter a three character name or a "Q" to quit the game”
3. Enter “asd” and press enter

Expected

1. The system should show the text “Your name is: “asd” on the second row.
2. Also “[]” on the first row, “You got 10 attempts left.” on the third row, a random number of underlines on the fourth row and “Guess a letter or enter "Q" to quit the game:” on the last row.



```
TERMINAL  ...  1: bash
[ ]
Your name is: asd
You got 10 attempst left.
_ _ _ _ _
Guess a letter or enter "Q" to quit the game:
█
```

Comments
Passed

TC 1.2 Name entered is to short.

This test is to test that the gamer can't enter a name that is less than 3 characters long.

Use Case: First five steps of UC2 "Play Game"

Scenario: The name is to short to be stored and the system ask the gamer for a new name.

Precondition: The app is running and user can see the menu.

Test steps

1. Press number 1 and press enter.
2. System shows "Enter a three character name or a "Q" to quit the game"
3. Enter "qw" and press enter

Expected

1. Should add the text "Too few characters!" in the console.

Comments
Passed

TC 1.3 Name entered is to long.

This test is to test that the gamer can't enter a name that is more than 3 characters long.

Use Case: First five steps of UC2 "Play Game"

Scenario: The name is to long to be stored.

Precondition: The app is running and user can see the menu.

Test steps

1. Press number 1 and press enter.
2. System shows "Enter a three character name or a "Q" to quit the game"
3. Enter "zxcv" and press enter

Expected

1. Should add the text "Too many characters!"in the console.

Comments
Passed

TC 2.1 Quit the game from menu.

Testing that the gamer can quit the game from the menu

Use Case: Use Case 3 Quit Game

Scenario: The gamer quits the game from the menu.

Precondition: The app is running and user can see the menu.

Test steps

1. Enter number 2 and press enter.
2. System ask for confirmation with the text “Are you sure you want to quit? (y/n):”
3. Enter lowercase “y” and press enter.

Expected

1. The whole game should be terminated.

Comments
Passed

TC 2.2 Quit the game instead of entering name.

Testing that the gamer can quit the game from “enter name” state.

Use Case: Use Case 3 Quit Game

Scenario: The gamer quits the game instead of entering the name.

Precondition: The app is running and user can see the menu.

Test steps

1. Enter number 1 and press enter
2. System shows “Enter a three character name or a "Q" to quit the game”
3. Enter uppercase “Q” and press enter.
4. System ask for confirmation with the text “Are you sure you want to quit? (y/n):”
5. Enter lowercase “y” and press enter

Expected

1. The whole game should be terminated.

Comments
Passed

TC 2.3 Quit the game from in the actual game

Testing that the gamer can quit the game from the main part of the game where the gamer can guess the letters of the word chosen.

Use Case: Use Case 3 Quit Game

Scenario: The gamer quits the game instead of guessing a letter.

Precondition: The app is running and user can see the menu.

Test steps

1. Enter number 1 and press enter
2. System shows “Enter a three character name or a "Q" to quit the game”
3. Enter “asd” and press enter
4. System ask gamer to guess a letter with the text ”Guess a letter or enter "Q" to quit the game:”
5. Enter uppercase “Q” and press enter.
6. System ask for confirmation with the text “Are you sure you want to quit? (y/n):”
7. Enter lowercase “y” and press enter

Expected

1. The whole game should be terminated.

Comments
Passed

TC 3.1 Return to menu.

Testing that the gamer can return to the menu from the high-score list.

Use Case: UC4 Check High-Scores

Scenario: The gamer has checked the high-score and wants to return to menu.

Precondition: The app is running and user can see the menu.

Test steps

1. Enter number 2 and press enter
2. System shows “HIGH SCORES!!”, the content of highscores.json and ”Enter "M" for menu or "Q" to quit the game”
3. Enter uppercase “M” and press enter.

Expected

1. The app should return to the menu.

Comments
Passed

TC 3.2 Quit Game from high-scores list.

Testing that the gamer can quit the game from the high-score list.

Use Case: UC4 Check High-Scores

Scenario: The gamer has checked the high-score and wants to quit the game.

Precondition: The app is running and user can see the menu.

Test steps

1. Enter number 2 and press enter
2. System shows “HIGH SCORES!!”, the content of highscores.json and ”Enter "M" for menu or "Q" to quit the game”
3. Enter uppercase “Q” and press enter.
4. System ask for confirmation with the text “Are you sure you want to quit? (y/n):”
5. Enter lowercase “y” and press enter

Expected

1. The app should terminate.

Comments
Passed

Manual Test Report

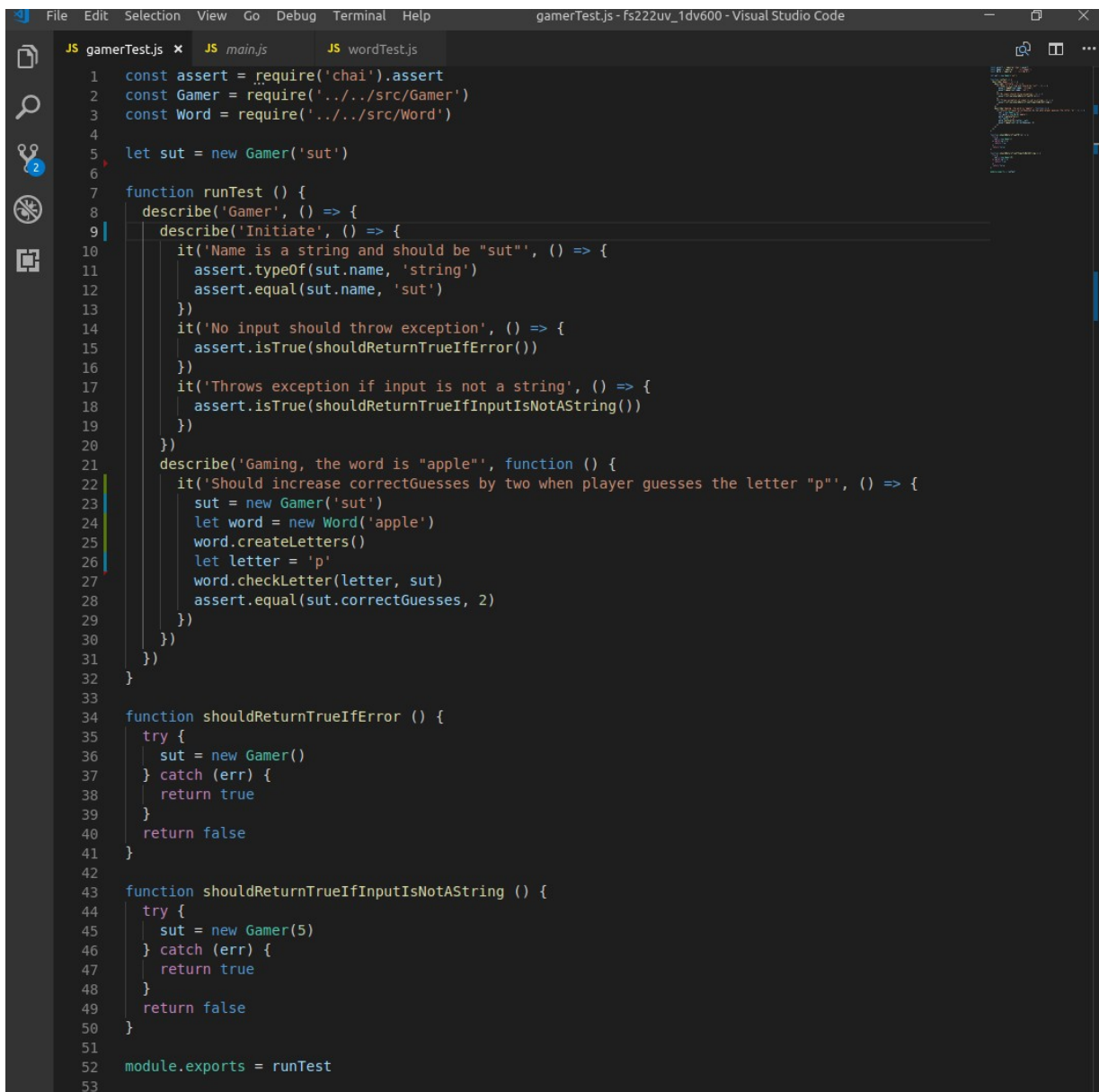
Test	UC2	UC3	UC4
TC1.1	1/OK	0	0
TC1.2	1/OK	0	0
TC1.3	1/OK	0	0
TC2.1	0	1/OK	0
TC2.2	0	1/OK	0
TC2.3	1/OK	1/OK	0
TC3.1	0	0	1/OK
TC3.2	0	1/OK	1/OK
COVERAGE & SUCCESS	4/OK	4/OK	2/OK

Automated Tests

Gamer

The first test is of the gamer class initiation. Here I test that it can take an input and put it into the name variable and that it should be a string. Also test that if the class is initiated without an input or that the input is something else than a string, it should throw an exception.

The second part I test that if the gamer guesses a letter correctly (or in this case, two letters), the correctGuesses counter should increase with the number of correct letters.



```
1  const assert = require('chai').assert
2  const Gamer = require('../src/Gamer')
3  const Word = require('../src/Word')
4
5  let sut = new Gamer('sut')
6
7  function runTest () {
8    describe('Gamer', () => {
9      describe('Initiate', () => {
10        it('Name is a string and should be "sut"', () => {
11          assert.typeOf(sut.name, 'string')
12          assert.equal(sut.name, 'sut')
13        })
14        it('No input should throw exception', () => {
15          assert.isTrue(shouldReturnTrueIfError())
16        })
17        it('Throws exception if input is not a string', () => {
18          assert.isTrue(shouldReturnTrueIfInputIsNotAString())
19        })
20      })
21      describe('Gaming, the word is "apple"', function () {
22        it('Should increase correctGuesses by two when player guesses the letter "p"', () => {
23          sut = new Gamer('sut')
24          let word = new Word('apple')
25          word.createLetters()
26          let letter = 'p'
27          word.checkLetter(letter, sut)
28          assert.equal(sut.correctGuesses, 2)
29        })
30      })
31    })
32  }
33
34  function shouldReturnTrueIfError () {
35    try {
36      sut = new Gamer()
37    } catch (err) {
38      return true
39    }
40    return false
41  }
42
43  function shouldReturnTrueIfInputIsNotAString () {
44    try {
45      sut = new Gamer(5)
46    } catch (err) {
47      return true
48    }
49    return false
50  }
51
52  module.exports = runTest
53
```


Word

This is all the tests for the word class method createLetters. Here I test if all the letters in the word array are independent objects and that there are actually one letter object for each letter in the chosen word (in this case the word is “apple”).

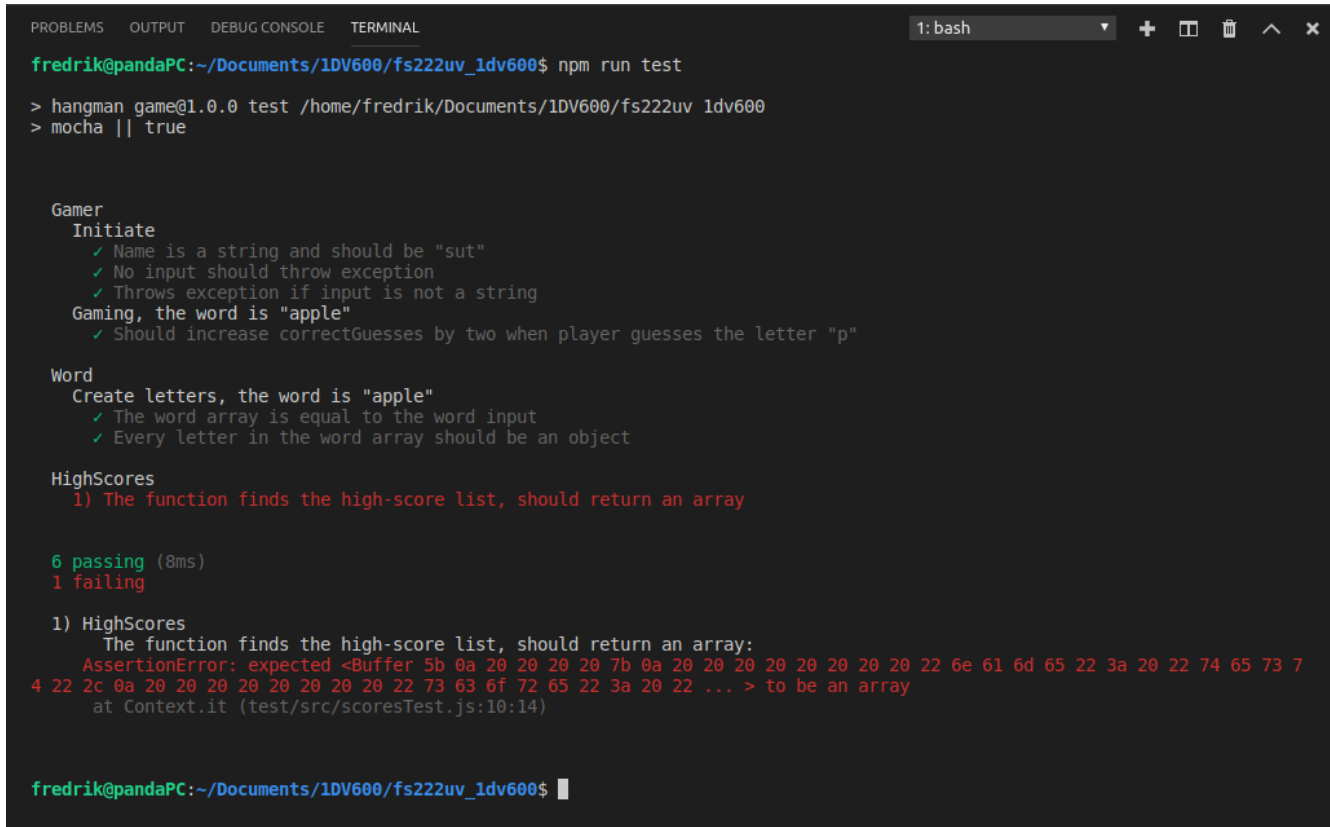
```
File Edit Selection View Go Debug Terminal Help wordTest.js - fs222uv_1dv600 - Visual Studio Code

JS wordTest.js x JS Letter.js

1 const assert = require('chai').assert
2 const Word = require('../src/Word')
3 const LetterClass = require('../src/Letter')
4
5 let sut = new Word('apple')
6
7 function runTest () {
8   describe('Word', () => {
9     describe('Create letters, the word is "apple"', () => {
10       it('The word array is equal to the word input', () => {
11         assert.isTrue(shouldReturnTrueIfWordArrayIsEqualToChosenWord())
12       })
13       it('Every letter in the word array should be an object', () => {
14         assert.isTrue(shouldReturnTrueIfAllLettersAreObjects())
15       })
16     })
17   })
18 }
19
20 function shouldReturnTrueIfWordArrayIsEqualToChosenWord () {
21   sut.createLetters()
22   let testArr = []
23   sut.wordArr.forEach(letter => {
24     testArr.push(letter.letter)
25   })
26
27   if (testArr.join('') === sut.chosenWord) {
28     return true
29   } else {
30     return false
31   }
32 }
33
34 function shouldReturnTrueIfAllLettersAreObjects () {
35   let numberOfObjects = 0
36   sut.createLetters()
37   sut.wordArr.forEach(obj => {
38     if (typeof obj === 'object') {
39       numberOfObjects++
40     }
41   })
42
43   if (numberOfObjects === sut.wordArr.length) {
44     return true
45   } else {
46     return false
47   }
48 }
49
50 module.exports = runTest
51
```

Results

This is all the results for all automated tests:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL 1: bash
fredrik@pandaPC:~/Documents/1DV600/fs222uv_1dv600$ npm run test
> hangman game@1.0.0 test /home/fredrik/Documents/1DV600/fs222uv 1dv600
> mocha || true

Gamer
  Initiate
    ✓ Name is a string and should be "sut"
    ✓ No input should throw exception
    ✓ Throws exception if input is not a string
  Gaming, the word is "apple"
    ✓ Should increase correctGuesses by two when player guesses the letter "p"

Word
  Create letters, the word is "apple"
    ✓ The word array is equal to the word input
    ✓ Every letter in the word array should be an object

HighScores
  1) The function finds the high-score list, should return an array

6 passing (8ms)
1 failing

1) HighScores
  The function finds the high-score list, should return an array:
    AssertionError: expected <Buffer 5b 0a 20 20 20 20 7b 0a 20 20 20 20 20 20 22 6e 61 6d 65 22 3a 20 22 74 65 73 7
4 22 2c 0a 20 20 20 20 20 20 20 20 22 73 63 6f 72 65 22 3a 20 22 ... > to be an array
    at Context.it (test/src/scoresTest.js:10:14)

fredrik@pandaPC:~/Documents/1DV600/fs222uv_1dv600$
```

Reflection

This was the most fun part of all the themes so far, learning how to use a testing framework and writing my own tests was fun and gave me a good learning experience. The easiest part was to write all the manual test, because this is the way I've been testing my apps to this point. The hardest parts was to come up with all different tests when the application is so small, and I noticed that I hadn't really written my code so that it would be easy to do meaningful automated tests.